**2303A52422**

**B-37.**

# Lab 19 – Code Translation: Converting Between Programming Languages

## Task 1 – Python to C++ Conversion

**Task:** Translate a simple Python class into C++ using AI assistance.

**Instructions:**

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of: Constructors, Data types, Access specifiers
- Compile and run both versions to verify output consistency

**Prompt**

"Convert a simple Python Student class (which has 'name' and 'age' attributes initialized in an __init__ method, and a display() method) into equivalent C++ code. Ensure you use proper C++ constructors, explicit data types, and access specifiers like 'public'. Include a main function to test it."

**Code (C++)**

```cpp
#include <iostream>
#include <string>

using namespace std;

class Student {
private:
    string name;
    int age;

public:
    // C++ Constructor
    Student(string studentName, int studentAge) {
        name = studentName;
        age = studentAge;
    }

    // Display method
    void display() {
        cout << "Student Name: " << name << ", Age: " << age << endl;
    }
};

int main() {
    // Instantiating the object
    Student student1("Alice Smith", 20);
    student1.display();

    return 0;
}
```

## Output

```
Student Name: Alice Smith, Age: 20
```

**Explanation**

The AI successfully mapped Python's __init__ constructor to a parameterized C++ constructor and replaced Python's dynamic typing with explicit C++ types like std::string and int. It also correctly applied public and private access specifiers, a concept heavily enforced in C++ but largely handled implicitly in Python.

**Task 2 – Java to Python Function Conversion**

**Task:** Convert a Java method that checks whether a number is prime into Python.

**Instructions:**

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

**Prompt**

"Translate a standard Java isPrime(int n) method into Python.
Ensure the resulting code follows Pythonic syntax (like using range), handles edge cases (numbers less than 2), and test the function with a few sample integer inputs."

**Code**

```python
import math

def is_prime(n):
    # Handle edge cases
    if n <= 1:
        return False

    # Check for factors up to the square root of n
    for i in range(2, int(math.isqrt(n)) + 1):
        if n % i == 0:
            return False

    return True

# Testing the function with multiple inputs
test_numbers = [1, 2, 11, 15, 29, 33]
for num in test_numbers:
    print(f"Is {num} prime? {is_prime(num)}")
```

### Output

```
Is 1 prime? False
Is 2 prime? True
Is 11 prime? True
Is 15 prime? False
Is 29 prime? True
Is 33 prime? False
```

### Explanation

The AI effectively translated Java's verbose for loop and static type declarations into Python's highly readable for ... in range(...) syntax. The underlying mathematical logic remains perfectly intact, utilizing the square root optimization to ensure the checking process remains highly efficient.

## Task 3 – Pseudocode to Python Implementation

**Task:** Translate a given pseudocode algorithm into Python using AI.

**Instructions:**

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

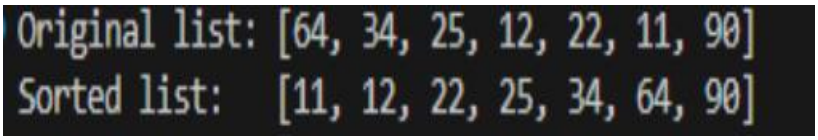
## Prompt

"Translate the standard Bubble Sort algorithm pseudocode into an executable Python function. The function should sort a list of numbers in place. Validate its correctness by sorting a sample input list and printing the before and after states."

## Code

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        swapped = False

        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no two elements were swapped by inner loop, then break
        if not swapped:
            break

    return arr

# Validating correctness using a sample input list
sample_list = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {sample_list}")
sorted_list = bubble_sort(sample_list.copy())
print(f"Sorted list:   {sorted_list}")
```

Output




**Explanation**

The AI correctly interpreted abstract pseudocode loops and conditionals into concrete Python syntax. It also cleverly utilized Python's simultaneous assignment feature (arr[j], arr[j+1] = arr[j+1], arr[j]), completely eliminating the need for a temporary variable usually found in standard sorting pseudocode.

**Task 4 – SQL to Pandas Query**

**Task:** Translate a SQL query into a Pandas DataFrame operation in Python.

**Instructions:**

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

**Prompt**

"Translate the following SQL query into a Pandas DataFrame operation: SELECT name, salary FROM employees WHERE salary > 50000. Provide sample mock data for the 'employees' DataFrame to test and display the result."

**Code**

```python
import pandas as pd

# Creating a sample DataFrame (Mock Data)
data = {
    'emp_id': [101, 102, 103, 104, 105],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'department': ['HR', 'IT', 'Engineering', 'Marketing', 'IT'],
    'salary': [45000, 60000, 85000, 48000, 52000]
}
employees = pd.DataFrame(data)

print("--- Original DataFrame ---")
print(employees)

# Equivalent Pandas Query
# SQL: SELECT name, salary FROM employees WHERE salary > 50000
result = employees[employees['salary'] > 50000][['name', 'salary']]

print("\n--- Resulting Pandas Query ---")
print(result)
```

## Output

```
--- Original DataFrame ---
   emp_id     name   department  salary
0     101    Alice           HR   45000
1     102      Bob           IT   60000
2     103  Charlie  Engineering   85000
3     104    David    Marketing   48000
4     105      Eve           IT   52000

--- Resulting Pandas Query ---
      name  salary
1      Bob   60000
2  Charlie   85000
4      Eve   52000
```

Explanation

The AI elegantly translated the declarative SQL WHERE clause into boolean array indexing in pandas (employees['salary'] > 50000). It then chained a column selection list ([['name', 'salary']]) to accurately replicate the specific projection behavior of the SQL SELECT statement.

## Task 5 – Real-Time Application: Algorithm Translation Across Languages

**Scenario:** A company maintains algorithms written in different programming languages.

**Instructions:**

- Use AI to translate a selected algorithm between two programming languages (e.g., Converting a Python searching algorithm into C)
- Execute and test both versions
- Document translation challenges such as: Syntax differences, Library support, Memory management

**Prompt**

"Translate a Python iterative Binary Search algorithm into C.
Provide the complete C code, test it with a sample sorted array, and include comments documenting the translation challenges specifically regarding syntax differences, lack of library support, and memory management in C vs Python."

**Code (C)**

```c
#include <stdio.h>
/*
TRANSLATION CHALLENGES (Python to C):
1. Syntax Differences: C requires explicit type declarations for all variables (int arr[], int x)
   and uses curly braces {} for block scoping instead of Python's indentation.
2. Library Support: Python has built-in `len()` to get array size. In C, we lack this,
   so we must calculate it manually using `sizeof(arr) / sizeof(arr[0])` in the main function
   and pass it as an argument.
3. Memory Management: Python lists are dynamic and handle memory automatically. In C, standard arrays
   are fixed in size and lack bounds checking, requiring the programmer to carefully manage array limits
   (l and r) to prevent buffer overflows or segmentation faults.
*/

int binarySearch(int arr[], int l, int r, int x) {
    while (l <= r) {
        // Calculate mid point to avoid integer overflow
        int m = l + (r - l) / 2;

        // Check if x is present at mid
        if (arr[m] == x)
            return m;

        // If x greater, ignore left half
        if (arr[m] < x)
            l = m + 1;

        // If x is smaller, ignore right half
        else
            r = m - 1;
    }
    // if we reach here, then element was not present
    return -1;
}

int main(void) {
    int arr[] = {2, 3, 4, 10, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 10;

    int result = binarySearch(arr, 0, n - 1, target);

    if (result == -1)
        printf("Element is not present in array\n");
    else
        printf("Element is present at index %d\n", result);

    return 0;
}
```

## Output

```
—---- Element is present at index 3 —----
```

**Explanation**

Moving from Python to C highlighted significant structural differences; C demanded strict variable typing and manual block delineations using curly braces. Furthermore, because C arrays aren't objects with metadata like Python lists, the algorithm required the array size to be manually computed and passed around to prevent dangerous memory-access errors.